

CAPÍTULO 3. LENGUAJE DE DEFINICIÓN DE DATOS (DDL)

El Lenguaje de Definición de Datos (<i>Data Definition Language, DDL</i>) permite definir el esquema de la base de datos, por medio de una serie de definiciones que se expresan en un lenguaje especial. El lenguaje más utilizado es SQL. Existen cuatro operaciones básicas: CREATE, ALTER, DROP y TRUNCATE El resultado de estas definiciones se almacena en el <i>Diccionario de Datos</i>.	<table border="1"> <tr> <td>CREATE</td><td>Permite crear objetos de datos, como nuevas bases de datos, tablas, vistas y procedimientos almacenado</td></tr> <tr> <td>ALTER</td><td>Permite modificar la estructura de una tabla u objeto. Se pueden agregar/quitar campos a una tabla, modificar el tipo de un campo, agregar/quitar índices a una tabla, modificar un trigger, etc.</td></tr> <tr> <td>DROP</td><td>Este comando elimina un objeto de la base de datos. Puede ser una tabla, vista, índice, trigger, función, procedimiento o cualquier objeto que el motor de la base de datos soporte.</td></tr> <tr> <td>TRUNCATE</td><td>Este comando trunca todo el contenido de una tabla. Mucho más rápido que DROP, para tablas grandes. Sólo sirve para eliminar todos los registros, no permite la cláusula WHERE. Internamente borra la tabla y la vuelve a crear y no ejecuta ninguna transacción.</td></tr> </table>	CREATE	Permite crear objetos de datos , como nuevas bases de datos, tablas, vistas y procedimientos almacenado	ALTER	Permite modificar la estructura de una tabla u objeto. Se pueden agregar/quitar campos a una tabla, modificar el tipo de un campo, agregar/quitar índices a una tabla, modificar un trigger, etc.	DROP	Este comando elimina un objeto de la base de datos. Puede ser una tabla, vista, índice, trigger, función, procedimiento o cualquier objeto que el motor de la base de datos soporte.	TRUNCATE	Este comando trunca todo el contenido de una tabla . Mucho más rápido que DROP, para tablas grandes. Sólo sirve para eliminar todos los registros, no permite la cláusula WHERE. Internamente borra la tabla y la vuelve a crear y no ejecuta ninguna transacción.
CREATE	Permite crear objetos de datos , como nuevas bases de datos, tablas, vistas y procedimientos almacenado								
ALTER	Permite modificar la estructura de una tabla u objeto. Se pueden agregar/quitar campos a una tabla, modificar el tipo de un campo, agregar/quitar índices a una tabla, modificar un trigger, etc.								
DROP	Este comando elimina un objeto de la base de datos. Puede ser una tabla, vista, índice, trigger, función, procedimiento o cualquier objeto que el motor de la base de datos soporte.								
TRUNCATE	Este comando trunca todo el contenido de una tabla . Mucho más rápido que DROP, para tablas grandes. Sólo sirve para eliminar todos los registros, no permite la cláusula WHERE. Internamente borra la tabla y la vuelve a crear y no ejecuta ninguna transacción.								
Sentencia CREATE DATABASE <pre>CREATE {DATABASE SCHEMA} [IF NOT EXISTS] db_name [create_specification [, create_specification] ...] create_specification: [DEFAULT] CHARACTER SET charset_name [DEFAULT] COLLATE collation_name</pre> CREATE SCHEMA puede usarse desde MySQL 5.0.2.									
CHARACTER SET y COLLATION MySQL Server admite varios juegos de caracteres, incluidos varios juegos de caracteres Unicode. <pre>SHOW CHARACTER SET; SHOW CHARACTER SET LIKE 'utf%'; SHOW COLLATION WHERE Charset = 'utf8mb4';</pre> SHOW collation where charset= 'utf8mb4' and collation like 'utf8mb4_sp%'; -muestra los españoles CHARSET utf8mb4 COLLATE utf8mb4_0900_ai_ci ■ Por defecto en MySQL	<pre>CREATE DATABASE IF NOT EXISTS nombre_BD DEFAULT CHARACTER SET latin1 COLLATE latin1_spanish_ci ; CREATE SCHEMA IF NOT EXISTS nombre_BD DEFAULT CHARACTER SET latin1 COLLATE latin1_spanish_ci ;</pre> La cláusula CHARACTER SET especifica el conjunto de caracteres por defecto de la base de datos. La cláusula COLLATE especifica la colación por defecto de la base de datos. Una de las principales diferencias es que CHARSET hace referencia a cómo MySQL guarda internamente los datos (es un conjunto de símbolos y codificaciones) y COLLATION es el conjunto de reglas que se aplican para comparar caracteres en un charset, es decir, indica a la base de datos como debe comparar los datos.								
Una COLLATION spanish_ci ordenará los registros de una forma mientras que spanish2_ci la ordenará de otra, debido a que el español tradicional considera "ch" como una letra entre la "C" y la "D". Asimismo, considera el uso de la letra "LL" como una letra entre la "L" y "M".	Podemos copiar toda la tabla y todos los datos <pre>CREATE TABLE nombre_tabla_nueva SELECT * FROM nombre_tabla_original</pre> Podemos crear la tabla con sólo algunos campos. <pre>CREATE TABLE libros_copia2 SELECT * FROM libros;</pre>								
CREATE TABLE nombre_tabla_nueva LIKE nombre_tabla_original. Crea la tabla vacía, pero mantiene la estructura exactamente igual. EXCEPTO CLAVES FORÁNEAS. Créate table libros2 like libros; Podemos copiarle todos los registros como sigue: <pre>Insert into libros2 select * from libros;</pre> Podemos insertar sólo algunos registros de la tabla. <pre>Insert into libros2 select * from libros where codigo<=3;</pre>	<pre>CREATE TABLE libros_copia3 SELECT codigo, autor FROM libros;</pre> La sentencia de creación CREATE TABLE (..) SELECT nos permite crear la tabla con los registros que devuelva la consulta de selección, pero tiene las siguientes limitaciones: • No traspasa las constraints de tipo PRIMARY KEY • No traspasa las definiciones de AUTO_INCREMENT • Utiliza el engine por defecto y no el de la tabla (en caso de que sean distintos) • Solamente traspasa los registros afectados por la sentencia SELECT, que podría no devolver ninguno y crear la tabla vacía.								
Una tabla TEMPORARY visible sólo para la conexión actual, se borra automáticamente cuando la conexión se cierra. Dos conexiones distintas pueden usar el mismo nombre de tabla temporal sin entrar en conflicto entre ellas ni con tablas no TEMPORARY con el mismo nombre. (La tabla existente se oculta hasta que se borra la tabla temporal.).	<pre>CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name [(create_definition,...)] [table_options] [select_statement]</pre> O: <pre>CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name [(()) LIKE old_tbl_name []];</pre>								

CAPÍTULO 3. LENGUAJE DE DEFINICIÓN DE DATOS (DDL)

Motores de tablas: Las opciones de tabla se utilizan para optimizar el comportamiento de la tabla. No es necesario que especifique ninguno de ellos.

DROP {DATABASE | SCHEMA} [IF EXISTS] *db_name*
DROP SCHEMA puede usarse desde MySQL 5.0.2.

DROP [TEMPORARY] **TABLE** [IF EXISTS]
tbl_name [, *tbl_name*] ...

IF EXISTS se usa para evitar un error si no existe. Usar **TEMPORARY** es la forma de asegurar que no borra accidentalmente una tabla no **TEMPORARY**.

Claves foráneas:

[**CONSTRAINT** [*nombreFK*] **FOREIGN KEY**
[index_name] (*col_name*, ...)
REFERENCES *tbl_name* (*col_name*,...)
[**ON DELETE** *reference_option*]
[**ON UPDATE** *reference_option*]

reference_option:

RESTRICT | **CASCADE** | **SET NULL** | **NO ACTION** | **SET DEFAULT**

Sintaxis básica:

FOREIGN KEY (campoForaneoFK) **REFERENCES**
nombre_tabla (*nombre_campo_PK*)

Si la clave foránea está compuesta por varios campos (clave foránea compuesta) o queremos ponerle un nombre lo indicaremos de la siguiente forma:

CONSTRAINT *FK_personas* **FOREIGN KEY** (*dep*, *id*)
REFERENCES *departamentos*(*dep*,*id*).

Condiciones y restricciones:

- Las tablas padre e hijo deben utilizar el mismo motor de almacenamiento y no se pueden definir como tablas temporales.
- Las columnas de la clave externa y la clave Foránea deben tener tipos de datos iguales.
- MySQL requiere índices en claves externas y claves Foráneas para que las comprobaciones puedan ser rápidas y no requieran un escaneo de tabla. En la tabla de referencia, debe haber un índice donde las columnas de clave externa se enumeran como las *primeras* columnas en el mismo orden.
- No se admiten prefijos de índice en columnas de clave externa. En consecuencia, las columnas **BLOB** y **TEXT** no se pueden incluir en una clave externa porque los índices de esas columnas siempre deben incluir una longitud de prefijo.
- Una tabla en una relación de clave externa no se puede modificar para utilizar otro motor de almacenamiento. Para cambiar el motor de almacenamiento, primero debe eliminar cualquier restricción de clave externa.
- Una restricción de clave externa no puede hacer referencia a una columna virtual generada.

Motores de almacenamiento

InnoDB	Tablas de transacciones seguras con bloqueo de filas y claves externas. El predeterminado para tablas nuevas.
MyISAM	Portátil binario que se usa principalmente para cargas de trabajo de solo lectura o principalmente de lectura.
MEMORY	Los datos se almacenan solo en la memoria.
CSV	Tablas que almacenan filas en formato de valores separados por comas.
ARCHIVE	El motor de almacenamiento de archivos.
EXAMPLE	Un motor de ejemplo.
FEDERATED	Motor que accede a tablas remotas.
HEAP	Este es un sinónimo de MEMORY.
MERGE	Una colección de MyISAM tablas utilizadas como una sola tabla. También conocido como MRG_MyISAM.
NDB	Tablas agrupadas, tolerante a fallas, basadas en memoria, transacciones de soporte y claves externas. También conocido como NDBCLUSTER.

reference_option:

RESTRICT | **CASCADE** | **SET NULL** | **NO ACTION** | **SET DEFAULT**

RESTRICT: Rechaza la operación de eliminación o actualización de la tabla principal. Especificar **RESTRICT**(o **NO ACTION**) *es lo mismo que omitir la cláusula ON DELETE o ON UPDATE*.

NO ACTION: **NO ACTION** *es lo mismo que RESTRICT*.

CASCADE: Elimina o actualiza la fila de la tabla principal y elimina o actualiza automáticamente las filas coincidentes en la tabla secundaria

SET NULL: Elimina o actualiza la fila de la tabla principal y establezca la columna o columnas de clave externa en la tabla secundaria en NULL. Se admiten ambas cláusulas **ON DELETE SET NULL** y **ON UPDATE SET NULL**.

*Si especifica una acción **SET NULL**, *asegúrese de no haber declarado las columnas en la tabla secundaria como NOT NULL*.

SET DEFAULT: Esta acción es reconocida por el analizador de MySQL, pero ambos **InnoDB** y **NDB** *rechazan definiciones de tablas que contienen cláusulas ON DELETE SET DEFAULT o ON UPDATE SET DEFAULT*.

CHECK Constraints:

Restricciones en la introducción de valores en una tabla, para mantener la integridad de la BBDD. 2 formatos:

- CHECK** (*expr*)
- [**CONSTRAINT** [*symbol*]] **CHECK** (*expr*) [[**NOT**] **ENFORCED**]

Symbol especifica un nombre para la restricción.

Expr especifica la condición de restricción.

- Si se omite o se especifica como **ENFORCED** (por defecto, *es igual que no poner nada*), si la restricción no se cumple da un ERROR.
- Si se especifica como **NOT ENFORCED**, si la restricción no se cumple ejecuta sin problemas y hace la inserción del registro.

CAPÍTULO 3. LENGUAJE DE DEFINICIÓN DE DATOS (DDL)

INDEX: Los índices son un grupo de datos vinculado a una o varias columnas que almacena una relación entre el contenido y la fila en la que se encuentra.
Los índices agilizan las **consultas** (SELECT) a la base de datos, pero la creación de índices implica un **aumento en el tiempo** de ejecución sobre aquellas consultas de **inserción, actualización y eliminación** realizadas sobre los datos afectados por el índice (ya que tendrán que **actualizarlo**). Hay que evitar un exceso de índices en tablas que sufran muchas modificaciones.

Normalmente, se crean todos los índices de una tabla cuando se crea la propia tabla con **CREATE TABLE**

Una lista de columnas (**col1,col2,...**) crea un índice de múltiples columnas (o **índice compuesto**).

Para columnas **CHAR** y **VARCHAR**, los índices pueden crearse para que usen sólo parte de una columna, usando **col_name(length)** para indexar un prefijo consistente en los primeros **length** caracteres de cada valor de la columna.

```
CREATE INDEX idx_nombre
```

```
ON Tabla_empleados (Nombre(10))
```

- Crea un índice compuesto

```
CREATE INDEX idx_nombre
```

```
ON Personas (nombre, apellidos);
```

ÍNDICES ÚNICOS

Previenen de la entrada de valores duplicados en columna index.

```
CREATE UNIQUE INDEX nombre_index
```

```
ON nombre_tabla (nombreCampo);
```

```
CREATE UNIQUE INDEX idx_dni
```

```
ON Personas (DNI);
```

ÍNDICES FULLTEXT

Permite realizar **búsquedas** dentro de un campo a partir de una cadena de caracteres.

```
CREATE FULLTEXT INDEX nombre_index
```

```
ON nombre_tabla (nombreCampo);
```

*Habitualmente, un método de **búsqueda sencillo** dentro de una tabla pasaba por utilizar LIKE:

```
SELECT texto FROM articulos WHERE texto LIKE '%palabra'
```

El problema aparecía cuando en vez de una sola palabra eran varias las que había que buscar:

Para esto sirve **FULLTEXT**: MySQL se encargará de comparar la cadena que le pasemos con los contenidos de la BD y devolver resultados aproximados.

Actualiza el campo fecha cada vez que se produce una actualización en el registro:

```
CREATE TABLE IF NOT EXISTS tabla_origen(  
id int auto_increment not null primary key,  
descripcion varchar(30),  
fecha timestamp default current_timestamp  
on update current_timestamp  
) engine = InnoDB; --Motor por defecto
```

CREATE INDEX: añade índices a tablas existentes

```
CREATE [UNIQUE|FULLTEXT|SPATIAL]
```

```
INDEX index_name
```

```
[USING index_type]
```

```
ON tbl_name (index_col_name,...)
```

```
index_col_name:
```

```
col_name [(length)] [ASC | DESC]
```

En MySQL hay cinco tipos de índices:

- **PRIMARY KEY:** Este índice se ha creado para generar consultas rápidas, debe ser único y no se admite el almacenamiento de valores NULL.
- **KEY** o **INDEX:** Son usados indistintamente por MySQL, permite crear índices sobre una columna, sobre varias columnas o sobre partes de una columna. (POR DEFECTO)
- **UNIQUE:** Este tipo de índice no permite el almacenamiento de valores duplicados.
- **FULLTEXT:** Permiten realizar búsquedas de palabras. Sólo pueden usarse sobre columnas **CHAR, VARCHAR** o **TEXT**
- **SPATIAL:** Este tipo de índices solo puede usarse sobre columnas de datos geométricos (spatial) y en el motor **MyISAM**, a partir de la versión 5.7 también en **InnoDB**.

ÍNDICES ESPACIAL

Contiene los datos espaciales, que representan la longitud, área y volumen de líneas, superficies y otros objetos. Se utiliza a menudo en el diseño asistido por ordenador, la elaboración de mapas y sistemas de información geográfica.

Permiten el tratamiento de datos en tres dimensiones dentro de una base de datos

```
CREATE ESPATIAL INDEX nombre_index
```

```
ON nombre_tabla (nombreCampo);
```

FUNCIONES Match y AGAINST

MATCH().

Esta función ejecuta la búsqueda de una cadena en una colección de texto (un conjunto de una o más columnas incluidas en un **índice FULLTEXT**).

AGAINST().

La cadena que se busca es dada como un argumento en la función AGAINST(), y es ejecutada en modo **case insensitive**.

```
SELECT id, titulo, contenido FROM articulos  
WHERE MATCH(titulo, contenido) AGAINST('Java');
```

DROP INDEX

Podemos eliminar un índice creado previamente

```
DROP INDEX nombre_index ON nombre_tabla;
```

CAPÍTULO 3. LENGUAJE DE DEFINICIÓN DE DATOS (DDL)

INNER JOIN

```
SELECT campo1, campo2, ...  
FROM table1  
INNER JOIN table2  
ON table1.column_name=table2.column_name;
```

```
SELECT campo1, campo2, ...  
FROM table1  
JOIN table2  
ON table1.column_name=table2.column_name;
```

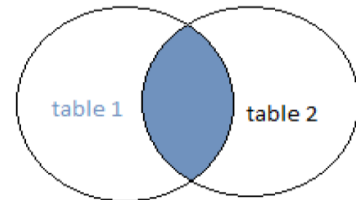
Si el campo de unión entre ambas tablas tiene el mismo nombre, podemos utilizar **USING**(campoComún)

```
SELECT campo1, campo2, ...  
FROM Tabla1  
INNER JOIN Tabla2 USING(CodigoComún);
```

La instrucción SQL JOIN se utiliza para combinar dos o más tablas, tomando un campo común de las dos.

El JOIN más común es: **SQL INNER JOIN (join simple)**. Un SQL INNER JOIN devuelve todos los registros de varias tablas que cumplen con la condición.

INNER JOIN



LEFT JOIN

```
SELECT campo1, campo2, ...  
FROM table1  
LEFT JOIN table2  
ON table1.column_name=table2.column_name;
```

La cláusula **LEFT JOIN** devuelve todos los registros de la tabla de la izquierda (table1), con las correspondientes de la tabla de la derecha (table2).

El resultado es NULL en la parte derecha cuando no hay registros que correspondan con la condición.

RIGHT JOIN

```
SELECT campo1, campo2, ...  
FROM table1  
RIGHT JOIN table2  
ON table1.column_name=table2.column_name;
```

```
SELECT campo1, campo2, ...  
FROM table1  
RIGHT OUTER JOIN table2  
ON table1.column_name=table2.column_name;
```

La instrucción **RIGHT JOIN** devuelve todos los registros de la tabla de la derecha (table2), y todos los registros correspondientes de la tabla de la izquierda (table1).

El resultado será NULL cuando no haya registros correspondientes de la tabla de la izquierda.

LEFT JOIN

RIGHT JOIN



JOIN ANIDADOS

```
SELECT *  
FROM tabla1  
INNER JOIN tabla2 ON tabla1.id=tabla2.id  
INNER JOIN tabla3 ON tabla2.id2=tabla3.id2  
[WHERE... GROUP BY ...ORDER BY...]
```

En esta consulta, los JOINS son ejecutados secuencialmente, primero se hace el primer JOIN, cuando está listo se hace el siguiente, luego el siguiente y así sucesivamente.

FORMULA DE LA EDAD

```
select alumnos.cod_alumno, alumnos.nombre  
Nombre_Alumno, apellido1, apellido2, FechaN,  
timestampdiff(year, FechaN, curdate()) edad  
From alumnos ;  
-- DA LA FECHA ACTUAL-- curdate()  
-- Me da los años:  
select timestampdiff(year, '2010-02-03', curdate());  
-- Me da en meses:  
select timestampdiff(month, '2010-02-03', curdate());  
-- Me da en días:  
select timestampdiff(day, '2010-02-03', curdate());  
  
select datediff(); -- da error por un día, hacer pruebas
```

-Muestra la edad de todos los alumnos de la siguiente forma: El alumno Pepe tiene 25 años:

```
select CONCAT('El alumno ', alumnos.nombre,  
'tiene ', timestampdiff(year, FechaN, curdate()), '  
años') as 'SOLUCION'  
from alumnos;
```